

MANYCHAT ALTERNATIVE · TECHNICAL GUIDE

The comment-to-DM flow on Instagram, step by step

Someone comments a word under your post and gets a private message with your resource. Here is every step to build it yourself with Meta's official APIs: the app, the receiver, the flow, the full code, and the tests to run before you switch it on.

Comment with the keyword → **Public reply** → **Message with a button** → **Follow** → **Link and question**

What you need

- An Instagram **professional** account (Business or Creator).
- A **second** Instagram account, only for testing.
- An account on **developers.facebook.com**.
- **Node.js 18** or later. The code is JavaScript; the logic works in any language.
- A public **https address** for the receiver: a tunnel is enough for testing (ngrok, Cloudflare Tunnel or similar), an always-on service for production.

How to read it

- Every step has a **CHECK** line: it tells you how to know it worked before moving on.
- Menu names in Meta's dashboard change often: if you can't find an item, search for its English name in the app settings.
- Calls contain **<PLACEHOLDERS>** to replace. The token and the App Secret are secrets: never put them in published code.

PART A · THE META APP

1 Create the app

Go to **developers.facebook.com**, log in and choose **My Apps → Create App**. As the use case pick the one for managing messages and content on Instagram and, if asked, the **Business** type. An app that isn't a Business app won't work: if you have an old one, create a new one.

CHECK you see the app dashboard, with **Instagram** in the left menu.

2 Open the Instagram setup

In the left menu: **Instagram → API setup with Instagram business login**. Meta also calls it *Instagram API with Instagram Login*: it's the simplest route and doesn't require a Facebook Page. Accounts, tokens and webhooks all live here.

3 Check the permissions

You need three:

```
instagram_business_basic
instagram_business_manage_comments
instagram_business_manage_messages
```

For your own account, and for anyone with a role on the app, **no Meta review is needed**: Standard access is enough. Review is only required if you use the app for other people's accounts.

4 Generate the token and get the IDs

On the same page, next to your account, click **Generate token**, log in to Instagram and copy the token: it lasts **60 days**. Then go to **App settings → Basic** and copy the **App Secret** (click Show). Finally ask Meta for your account ID:

```
GET https://graph.instagram.com/v25.0/me?fields=user_id,username&access_token=
<TOKEN>
```

CHECK the response contains your username and a **user_id**: that is your **IG_ID**, the same one you'll see as **id** in webhook notifications. Write down three values: **TOKEN**, **APP_SECRET**, **IG_ID**.

5 Add the second account as a tester

In the dashboard: **App roles → Roles → Add people → Instagram Tester**, and enter the second account's username. Then, **from the second account**, in Instagram: **Settings → Apps and websites → Tester invites** and accept.

Why it matters: with Standard access the app only sees people who have a role on the app. Without this step the second account's comments never arrive, and you'll think everything else is wrong.

CHECK in the dashboard the second account shows up among the testers, invite accepted.

PART B · THE RECEIVER

6 Put the receiver online

The code is in part D. Start it and make it reachable from a public **https** address: a tunnel to port 3000 for testing, an always-on service in production. The variables to set are at the end of part D.

CHECK opening the address in a browser, the server answers (even just "no"): it means it's reachable.

7 Register the webhook

On the page from step 2 find **Configure webhooks** and fill in:

- **Callback URL**: the receiver's https address.
- **Verify token**: a string of your choice, the same one you set in **VERIFY_TOKEN**.

Click **Verify and save**. Meta sends a GET request to your address with **hub.mode**, **hub.verify_token** and **hub.challenge**: the server must answer with the value of **hub.challenge**. The code already does it.

CHECK the dashboard saves without errors.

8 Subscribe your account to the events

```
POST https://graph.instagram.com/v25.0/me/subscribed_apps
     ?subscribed_fields=comments,messages,messaging_postbacks
     &access_token=<TOKEN>
```

In the dashboard, in the same webhooks section, also turn on the **comments**, **messages** and **messaging_postbacks** fields.

CHECK the response is `{"success": true}`. With `GET .../me/subscribed_apps` you see the subscribed fields.

The 4 receiver rules, according to Meta

- Answer **200 right away** to every notification, then do the work. If you don't, Meta retries for **36 hours**.
- **Drop duplicates**: because of retries the same notification can arrive more than once.
- **Verify the signature**: the **X-Hub-Signature-256** header is an HMAC SHA256 of the **raw** body, computed with the App Secret. If it doesn't match, discard.
- Notifications arrive **in batches** (up to 1000): loop over every item, don't just read the first.

PART C · THE FLOW, CALL BY CALL

9 The comment arrives

Meta sends the receiver a **comments** notification. You need four values: the **comment id**, the **commenter's id**, the **text** and the **post id**.

```
{ "object": "instagram", "entry": [ { "id": "<IG_ID>", "time": 1700000000,
  "changes": [ { "field": "comments", "value": {
    "id": "<COMMENT_ID>",
    "from": { "id": "<PERSON_ID>", "username": "name" },
    "text": "I want the GUIDE",
    "media": { "id": "<MEDIA_ID>", "media_product_type": "REELS" } } } ] } ] }
```

Check whether the text contains your keyword (whole word, case-insensitive) and handle **only each person's first comment** on that post.

10 Public reply, under the comment

```
POST https://graph.instagram.com/v25.0/<COMMENT_ID>/replies
Authorization: Bearer <TOKEN>
{ "message": "Done, check your DMs" }
```

It only works on top-level comments. Rotate 3 to 5 variants.

11 Private reply: the first message

You send it with the **comment id** in place of the person's id. Meta allows **only one per comment**, within **7 days**. It lands in the inbox if the person follows you, in the **Requests** folder if they don't, with a link to the post.

Try 1 · with the button (this is what you want: the tap opens the 24 hours):

```
POST https://graph.instagram.com/v25.0/<IG_ID>/messages
Authorization: Bearer <TOKEN>
{ "recipient": { "comment_id": "<COMMENT_ID>" },
  "message": { "attachment": { "type": "template", "payload": {
    "template_type": "button",
    "text": "Hey, here I am. I'm sending you the guide right now. Tap the button
and it's yours.",
    "buttons": [ { "type": "postback", "title": "Send it to me", "payload":
"LINK:GUIDE" } ] } } } }
```

> **To be tested, not guaranteed.** In Meta's documentation the private reply is described only with plain text, and the button only with the person's id in place of the comment id. Nowhere does it say they can be combined. The code in part D tries the button and, if Meta rejects it, falls back to try 2 on its own.

Try 2 · text only, and the person replies (always works):

```
{ "recipient": { "comment_id": "<COMMENT_ID>" },
  "message": { "text": "Hey, here I am. I'm sending you the guide right now. Reply
with the word GO and it's yours." } }
```

When the person replies, the receiver gets a **messages** notification and the 24 hours are open, just like with the tap.

CHECK from the second account the message arrives (check **Requests** too). Note which try worked: it's the most important piece of information in this whole guide.

12 The button tap

When the person taps, a **messaging_postbacks** notification arrives. From it you get their **id** (**sender.id**) and **24 hours** to write.

```
{ "object": "instagram", "entry": [ { "id": "<IG_ID>", "messaging": [ {
  "sender": { "id": "<PERSON_ID>" }, "recipient": { "id": "<IG_ID>" },
  "postback": { "mid": "...", "title": "Send it to me", "payload": "LINK:GUIDE" } }
] } ] }
```

The **payload** is the string you chose: here it tells you which keyword they're waiting on.

13 The follow

Before the link, ask them to follow you and check whether they do. The check is a call on the person's id:

```
GET https://graph.instagram.com/v25.0/<PERSON_ID>?fields=is_user_follow_business
Authorization: Bearer <TOKEN>      →  { "is_user_follow_business": false }
```

It only works after the person has tapped a button or replied. The way I recommend is **soft**: if they don't follow you, send the message with the request and a second button; **on the second tap the link goes out anyway**. The check is sometimes wrong for long-time followers, and blocking a real follower costs more than a link given away.

14 The link and the question

```
POST https://graph.instagram.com/v25.0/<IG_ID>/messages
Authorization: Bearer <TOKEN>
{ "recipient": { "id": "<PERSON_ID>" },
  "message": { "text": "Here's the guide: https://...\\n\\nWhich part do you want
to put into practice first?" } }
```

The link starts with **https**. The question is a single one, answerable in one line: it's what starts the conversation.

CHECK from the second account: comment, public reply, message, tap, follow request, second tap, link. If the whole chain comes back, the flow works.

PART D · THE FULL CODE

One file, **bot.mjs**, no libraries to install. It does the whole chain: verification, signature, duplicates, public reply, try 1 and try 2, soft follow, link. The rules (one row per keyword) are at the top: swap in your own.

```
import http from "node:http";
import crypto from "node:crypto";

const { VERIFY_TOKEN, APP_SECRET, IG_ID, IG_TOKEN, PORT = 3000 } = process.env;
const API = process.env.GRAPH || "https://graph.instagram.com/v25.0";

// 1. The rules: one row per keyword
const RULES = {
  GUIDE: {
    active: true,
    replies: ["I just sent you a message", "Done, check your DMs", "Sent, check your
messages"],
    dm: "Hey, here I am. I'm sending you the guide right now. Tap the button and it's
yours. (Automated message)",
    button: "Send it to me",
    follow: "Almost there! Follow me to catch the next videos, then tap the button
again.",
    link: "https://example.com/guide",
    question: "Which part do you want to put into practice first?",
  },
};

const seen = new Set();     // dedupe: comments already handled
const asked = new Set();    // who has already been asked to follow

const post = (path, body) =>
  fetch(`${API}${path}`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${IG_TOKEN}`
},
```

```
    body: JSON.stringify(body),
  }).then(async (r) => ({ ok: r.ok, status: r.status, json: await r.json().catch(() =>
({})) }));

const buttonMessage = (text, title, payload) => ({
  attachment: {
    type: "template",
    payload: { template_type: "button", text, buttons: [{ type: "postback", title,
payload }] },
  },
});

const send = (igsid, message) => post(`/${IG_ID}/messages`, { recipient: { id: igsid },
message });

function findKeyword(text) {
  const words = (text || "").toUpperCase().split(/[^A-Z0-9]+/);
  return Object.keys(RULES).find((k) => RULES[k].active && words.includes(k));
}

// 2. A comment arrives
async function handleComment(c) {
  if (seen.has(c.id) || c.from.id === IG_ID) return;
  seen.add(c.id);
  const keyword = findKeyword(c.text);
  if (!keyword) return;
  const key = `${c.from.id}:${c.media.id}`;           // only each person's first
comment
  if (seen.has(key)) return;
  seen.add(key);
  const r = RULES[keyword];

  await post(`/${c.id}/replies`, { message: r.replies[Math.floor(Math.random() *
r.replies.length)] });
```

```
  // TRY 1: private reply with the button
  let result = await post(`/${IG_ID}/messages`, {
    recipient: { comment_id: c.id },
    message: buttonMessage(r.dm, r.button, `LINK:${keyword}`),
  });
  // TRY 2: if Meta rejects the button, plain text asking them to reply
  if (!result.ok) {
    console.log("button rejected, using text:", result.status,
JSON.stringify(result.json));
    await post(`/${IG_ID}/messages`, {
      recipient: { comment_id: c.id },
      message: { text: `${r.dm.split(" Tap")[0]} Reply to this message with the word GO
and I'll send it.` },
    });
  }
}

// 3. A button tap arrives (or the text reply from TRY 2)
async function handleMessage(m) {
  const igsid = m.sender.id;
  if (igsid === IG_ID || m.message?.is_echo) return;
  let keyword, secondTap = false;
  if (m.postback) {
    const [type, k, n] = m.postback.payload.split(":");
    keyword = k; secondTap = n === "2";
    if (type !== "LINK") return;
  } else if (m.message?.text && /^\s*go\s*$/i.test(m.message.text)) {
    keyword = Object.keys(RULES)[0];
  } else return;
  const r = RULES[keyword];
  if (!r) return;

  // 4. The follow: soft check, on the second tap the link goes out anyway
```

```
  const profile = await fetch(`${API}/${igsid}?fields=is_user_follow_business`, {
    headers: { Authorization: `Bearer ${IG_TOKEN}` },
  }).then((x) => x.json()).catch(() => ({}));
  if (!profile.is_user_follow_business && !secondTap && !asked.has(igsid)) {
    asked.add(igsid);
    await send(igsid, buttonMessage(r.follow, r.button, `LINK:${keyword}:2`));
    return;
  }
  // 5. The link and the question
  await send(igsid, { text: `Here's the guide: ${r.link}\n\n${r.question}` });
}

// The receiver
http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://x");
  if (req.method === "GET") {                                  // Meta's verification
    const ok = url.searchParams.get("hub.mode") === "subscribe" &&
      url.searchParams.get("hub.verify_token") === VERIFY_TOKEN;
    res.writeHead(ok ? 200 : 403).end(ok ? url.searchParams.get("hub.challenge") :
"no");
    return;
  }
  const chunks = []; for await (const c of req) chunks.push(c);
  const raw = Buffer.concat(chunks);
  const expected = "sha256=" + crypto.createHmac("sha256",
APP_SECRET).update(raw).digest("hex");
  const received = req.headers["x-hub-signature-256"] || "";
  const valid = received.length === expected.length &&
    crypto.timingSafeEqual(Buffer.from(received), Buffer.from(expected));
  if (!valid) { res.writeHead(401).end(); return; }
  res.writeHead(200).end("ok");                                // answer right away,
then work
  try {
    for (const e of JSON.parse(raw).entry || []) {
```

```
      for (const ch of e.changes || []) if (ch.field === "comments") await
handleComment(ch.value);
      for (const m of e.messaging || []) await handleMessage(m);
    }
  } catch (err) { console.error(err); }
}).listen(PORT, () => console.log("listening on", PORT));
```

How to start it

```
export VERIFY_TOKEN="the-string-from-step-7"
export APP_SECRET="the-app-secret-from-step-4"
export IG_ID="the-id-from-step-4"
export IG_TOKEN="the-token-from-step-4"
node bot.mjs        # listening on port 3000
```

- **What has been verified:** the logic (signature, duplicates, whole-word match, the sequence of steps, the fallback when the button is rejected) is tested against a fake Meta server, 17 checks out of 17. **It has not been tested against Meta:** you do that with part E.
- **Limits of this file:** duplicates and who has already been asked to follow are kept in memory, and are lost on restart. For real use, store them in a file or a database.

PART E · THE TESTS, BEFORE YOU SWITCH IT ON

1. Publish (or use) a test post on your account and write the keyword in the caption.
2. From the **second account** comment with the keyword. Then watch the receiver's logs.
3. Check: public reply under the comment, private message (also in **Requests**), tap or reply, follow request, second tap, link.
4. Note **which try** from step 11 worked (button or text only) and adjust your texts accordingly.
5. Open the link **from your phone**: it must start with https and load.
6. Comment twice from the same account: the second time nothing should happen.
7. The keyword stays **off** (`active: false`) until the day you publish.

IF IT DOESN'T WORK

WHAT YOU SEE	WHAT TO CHECK
The dashboard won't save the webhook	The address isn't a public https one, the server doesn't answer with **hub.challenge**, or the **Verify token** differs from VERIFY_TOKEN.
The webhook is saved, but nothing arrives	Account not subscribed (step 8), fields not turned on, the commenter **isn't a tester** (step 5), account isn't professional, comment on someone else's post.
It arrives, but the bot answers 401	The signature. It must be computed on the **raw** body, not on re-serialized JSON, and with the App Secret of **this** app.
Error on the private reply	Comment older than 7 days, a message already sent for that comment, expired token or missing permission. Read the error code in Meta's response.
The button is rejected	That's the try 2 case: the code falls back to text. Look for the "button rejected" message in the logs.
You can't see the message	Look in the **Requests** folder: if the second account doesn't follow you, it lands there.
Duplicate messages	Deduplication is missing. Meta retries if your answer is slow: answer 200 before working.
It stops after a few weeks	The token lasts 60 days. Renew it before it expires (part F).

PART F · RULES, LIMITS AND PRODUCTION

Meta's limits

- One automated message per comment, within 7 days.
- To write again you need a reply or a tap: from then you have 24 hours.
- Up to **750 private replies per hour** per account, on comments to posts and reels.
- The message can't go out if the comment is hidden, or belongs to a live that has ended.
- You can only reply publicly to top-level comments.

Good habits

- **Say it's automated** in the first message. Meta requires it where the law does and recommends it everywhere.
- One rule per keyword, and the same keyword in the video and in the caption.
- Store only what you need (id, keyword, outcome) and say in your privacy notice what you keep.
- No messages to people who haven't written to you.

To keep it running for real

1. **An always-on service** for the receiver, with automatic restart if it crashes.
2. **A store** instead of memory: comments already handled and who has been asked to follow.
3. **Renew the token** before the 60 days are up (it must be at least 24 hours old and not expired; after renewal it's valid for another 60 days):

```
GET https://graph.instagram.com/refresh_access_token
    ?grant_type=ig_refresh_token&access_token=<TOKEN>
```

4. **A log** of who asked for what and whether the link arrived: it tells you which video brings in contacts.
5. If you want to use it for **other people's accounts** (clients), you need Meta's review: it's a separate job, with videos and permissions one by one.

Honesty, what this guide doesn't say

- **Meta's rules and APIs change.** The steps were checked against the official documentation on 29 Oct 2026 (pages updated between March and June 2026): re-check before you build.
- **The button inside the private reply is not documented by Meta.** That's why there are two tries. I can't tell you in advance which one will work on your account.
- **Menu names** in Meta's dashboard change: the ones given are my reading of the documentation, not a screenshot from today.
- **The follow:** asking people to follow before the resource isn't forbidden, but Meta doesn't expressly endorse it. Your call.
- **Regulated sectors**, like healthcare: Meta restricts using messaging to connect people with healthcare providers. Read its policies first.
- It covers **one path only**, from comment to resource. Longer flows or sends to a list aren't here.

Stuck on a step? I'll help you build it.

If the app, the receiver or the tests don't add up, book a 15-minute call and we'll look at it together.

Book a 15-minute call